

```
$ onboard mksglu/context-mode --profile dev
```

context-mode : un serveur MCP et des hooks qui gardent la sortie brute des outils hors du contexte de l'agent, actif et bien testé mais porté par un seul mainteneur : contribue petit, avec tests

Context window optimization for AI coding agents. Sandboxes tool output (98% reduction), persists session memory, and enforces routing across 17 platforms via MCP + hooks.

21 624

ÉTOILES GITHUB

10CONTRIBUTEURS ACTIFS
sur 12 semaines**107**

PR OUVERTES

8 sept. 2026

DERNIER COMMIT

Que fait ce projet et pour qui ?

> Un serveur MCP plus des hooks pour agents de code, 21624 étoiles et 1555 forks depuis le 2026-02-23, sous licence ELv2 non OSI : lis `LICENSE` avant tout usage en service managé ^{1 2 3 4 5}.

context-mode est un serveur MCP qui met la sortie brute des outils en sandbox hors de la fenêtre de contexte (réduction annoncée de 98 %), persiste une mémoire de session et impose le routage sur 17 plateformes via MCP + hooks ⁶.

Quatre axes dans le README : sandbox des outils, continuité de session stockée en SQLite, indexée FTS5 et cherchée en BM25, « think in code » (l'agent écrit un script qui `console.log` seulement le résultat au lieu de lire les fichiers un par un), et aucune contrainte de style sur la réponse finale ⁷.

Public : les développeurs qui travaillent avec un agent de code ; les topics couvrent claude-code, codex-cli, copilot, cursor-plugin, kiro, opencode, openclaw, pi-agent, zed-extension ⁸. Le README détaille l'installation par plateforme : plugin marketplace pour Claude Code, `~/.gemini/settings.json` pour Gemini CLI, `.vscode/mcp.json` + `.github/hooks/context-mode.json` pour VS Code Copilot ⁷.

Chiffres : 21624 étoiles, 1555 forks, 226 issues ouvertes, dernier push le 2026-09-08 ^{1 2 9 10}.

Dépôt créé le 2026-02-23 sur un compte utilisateur (pas une organisation), non archivé, branche par défaut `main` ^{3 11 12 13}.

Licence Elastic License 2.0 (ELv2), non OSI : lire `LICENSE` avant tout usage en service managé ou redistribution ^{4 5}.

Comment le code est-il organisé ?

> Tout le TypeScript est dans `src/` (serveur, CLI, adaptateurs), les hooks sont du JS pur dans `hooks/` sans build, et `start.mjs` charge `server.bundle.mjs` avant `build/` : `rm server.bundle.mjs` en dev local, sinon tes changements ne sont jamais chargés ^{14 15 16 17}.

- └ `.agents/` config agents IA
- └ `.claude-plugin/` manifeste plugin Claude Code
- └ `.claude/` config locale Claude Code
- └ `.codex-plugin/` manifeste plugin Codex
- └ `.cursor-plugin/` manifeste plugin Cursor
- └ `.github/` 5 workflows GitHub Actions
- └ `.openclaw-plugin/` manifeste plugin OpenClaw
- └ `.pi/` extension pour Pi
- └ `bin/` script `statusline.mjs`
- └ `configs/` configs par plateforme
- └ `docs/` documentation
- └ `hooks/` hooks par plateforme, bundles session
- └ `scripts/` scripts build, install, version
- └ `skills/` skills pour agents
- └ `src/` TypeScript : serveur MCP, CLI, adaptateurs
- └ `tests/` tests vitest

... et 20 autres entrées, voir `sources/tree.md`

`src/` : TypeScript du serveur MCP, de la CLI et des adaptateurs ; `tsc` compile vers `build/`, puis esbuild bundle `src/server.ts` en `server.bundle.mjs` et `src/cli.ts` en `cli.bundle.mjs` à la racine ^{14 18 19 20}.

Fichiers clés de `src/` : `server.ts` (serveur MCP, handlers d'outils), `store.ts` (FTS5), `executor.ts` (exécuteur polyglotte), `security.ts` (règles deny/allow), `session/{db,extract,snapshot}.ts`, `adapters/<plateforme>/` (claude-code, gemini-cli, opencode, codex, vscode-copilot, omp, qwen-code) ^{17 21}.

`hooks/` : hooks en JS pur (`.mjs`), aucun build : `pretooluse.mjs` (routage), `posttooluse.mjs` (capture d'événements), `precompact.mjs` (snapshot de reprise), `sessionstart.mjs`, `stop.mjs`, un sous-dossier par plateforme et des bundles `session-*.bundle.mjs` produits par esbuild ^{15 21 17}.

`configs/` : fichiers d'installation par plateforme, servis tels quels ; `tests/` : vitest ; `scripts/` : build, install, version ; `docs/` : adr/ , `adapters/` , `platform-support.md` ^{22 23 24 25}.

Un manifeste de plugin par client à la racine : `.claude-plugin/` , `.codex-plugin/` , `.cursor-plugin/` , `.openclaw-plugin/` , `.pi/` , `openclaw.plugin.json` ; `.github/` porte 5 workflows ^{26 27 28 29 30 31}.

`start.mjs` charge `server.bundle.mjs` s'il existe, sinon `build/server.js` : en dev local, `rm server.bundle.mjs`, sinon tes modifications de `src/` ne sont jamais chargées ^{16 17}.

Par où commencer à lire ?

```
> Ouvre src/server.ts (handlers des outils ctx_* ), puis src/adapters/opencode/index.ts comme modèle d'un HookAdapter , puis hooks/sessionstart.mjs
→ src/session/db.ts pour la continuité de session 32 33 17.
```

src/server.ts : le serveur MCP et ses handlers d'outils ; npm run dev = npx tsx src/server.ts ; bundlé en server.bundle.mjs ^{32 34}.

src/cli.ts : la CLI context-mode (bin → cli.bundle.mjs), commandes setup et doctor ; la CI lance npx tsx src/cli.ts doctor ^{35 36}.

src/adapters/opencode/index.ts : cible de main dans package.json, classe OpenCodeAdapter qui implémente HookAdapter , le modèle pour comprendre un adaptateur ^{33 37} ; l'interface HookAdapter est dans src/adapters/types.ts , la détection de plateforme dans src/adapters/detect.ts ¹⁷.

start.mjs (lanceur publié avec le paquet npm) et src/adapters/openclaw/index.ts (export ./openclaw) sont déduits du nom par le collecteur, non lus ^{38 39}.

Pour la continuité de session, lire dans l'ordre hooks/sessionstart.mjs → src/session/db.ts → src/session/snapshot.ts : SessionDB persistante dans ~/.claude/context-mode/sessions/<hash>.db , ContentStore éphémère dans /tmp/context-mode-<PID>.db , les événements bruts ne sont jamais injectés dans le contexte, le modèle les retrouve via ctx_search ¹⁷.

Pour la liste et la sémantique des outils ctx_* (sandbox : ctx_batch_execute , ctx_execute , ctx_execute_file , ctx_index , ctx_search , ctx_fetch_and_index ; méta : ctx_stats , ctx_doctor , ctx_upgrade , ctx_purge , ctx_insight), lire la section Install → Claude Code du README, puis leurs handlers dans src/server.ts ⁷.

Comment builder, tester et lancer ?

```
> npm install && npm run build && npm test avec node >= 22.5.0 : vitest sur 255 fichiers de test, et la CI rejoue la même chaîne sur ubuntu, macos et windows 40 41 42 43 44.
```

Prérequis : node >= 22.5.0 (`engines`), Bun accepté par le CONTRIBUTING ; `better-sqlite3` est un module natif externalisé du bundle ; `package.json` déclare `packageManager: pnpm` et le repo contient `bun.lock` , mais CONTRIBUTING et CI utilisent npm ^{43 20 17 45}.

Installer puis builder, dans cet ordre : `npm install` puis `npm run build` (tsc → `build/` , puis `npm run bundle` , `assert-bundle` , `assert-asymmetric-drift`) ^{40 17 20}.

Tester : `npm test` (= `vitest run` , `pretest` relance `npm run build`) ; `npm run typecheck` (`tsc --noEmit`) ; `npm run test:watch` ; 255 fichiers de test sous `tests/` , framework vitest ^{41 20 42 46 47}.

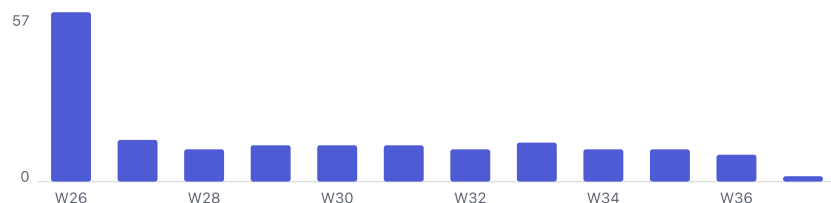
Lancer : en dev `npm run dev` ; en usage `claude mcp add context-mode -- npx -y context-mode` (MCP seul, sans hooks) ou `/plugin marketplace add mksglu/context-mode` puis `/plugin install context-mode@context-mode` (plugin complet), vérification par `/context-mode:ctx-doctor` ^{48 7}.

Dev local dans Claude Code : symlink de `~/claude/plugins/cache/context-mode/context-mode/<version>` vers ton clone, override du seul hook `PreToolUse` dans `~/claude/settings.json` (les autres sont déclarés dans `hooks/hooks.json`), `rm server.bundle.mjs` , `pkill -f "context-mode.*start.mjs"` , redémarrer Claude Code ; modifier `hooks/*.mjs` ou `configs/*` ne demande pas de rebuild, modifier `src/**/*.ts` oui ¹⁷.

CI `.github/workflows/ci.yml` (nom « CI ») : déclenchée par `workflow_dispatch`, `push` et `pull_request` sur `main` et `next` , matrice `ubuntu/macos/windows`, Node 22.5 plus Python, Go et Elixir, étapes `npm install` → `npx tsc -b --noEmit` → `npm run build` → `npm run bundle` → `npm run assert-bundle` → `vitest` → `npx tsx src/cli.ts doctor` ; les autres workflows (`bundle.yml`, `openclaw-e2e.yml`, `tier2-e2e-smoke.yml`, `update-stats.yml`) ne sont pas dans le cache : à vérifier dans `.github/workflows/` ^{49 50 51 52 53 54}.

Comment le projet vit-il ?

> Entre 9 et 14 commits par semaine depuis 2026-W27, mais bus factor 1 : mksglu signe 60 commits, le suivant 6, et 107 PR ouvertes attendent sa revue ^{55 56 57 58 59 60}.



Sur les 12 dernières semaines : pic à 57 commits en 2026-W26, puis entre 9 et 14 par semaine de W27 à W36, et 2 en W37 (semaine en cours) ; dernier commit le 2026-09-08 ^{61 55 56 62 63}.

Bus factor 1 : mksglu porte 60 commits, ken-jo 6, pg-adm1n 2, les autres contributeurs listés 1 chacun ; le mainteneur se décrit comme « solo maintain with limited time » ^{57 58 59 64 65 66 17}.

Releases : 10 tags dans le cache, de v1.0.160 (2026-06-01) à v1.0.169 (2026-06-29) ; le collecteur ne garde que les 10 dernières, le cache ne dit donc pas si une version est sortie depuis juin alors que les commits continuent : à vérifier sur <https://github.com/mksglu/context-mode/releases> ^{67 68 69 70}.

Flux entrant : 119 issues ouvertes, 6 fermées sur 30 jours, 107 PR ouvertes ; le cache ne liste aucune PR fusionnée sur 30 jours (liste vide), à vérifier sur <https://github.com/mksglu/context-mode/pulls?q=is%3Amerged> ^{71 72 60 73}.

Les PR ouvertes les plus récentes datent d'aujourd'hui et d'hier (#1138 fix(fetch), #1137 docs, #1129 fix(hooks) ABI) : les contributions externes arrivent, à mettre en regard d'un mainteneur seul pour la revue ^{74 75 76 77 17}.

Risques notés par le collecteur : CI low, deps low, licence mid, sécurité mid (pas de SECURITY.md), bus factor high ; pas de CODEOWNERS, un fichier de funding existe ^{78 79 80 81 82 83 84}.

Quelle première contribution ?

> Pas de good first issue : prends un bug ouvert et discuté (#947 timeout de `ctx_batch_execute` , #1024 nettoyage de DB), écris le test dans le fichier existant du domaine, joins `bash scripts/ctx-debug.sh` , et vérifie d'abord qu'aucune des 107 PR ouvertes ne le couvre ^{85 86 87 60 17}.

Aucune issue « good first issue » dans le cache (liste vide) ; labels présents : enhancement 6, bug 4, help wanted 1 ; l'entrée sans code est l'issue #45 « Beta testers wanted » (154 commentaires, label help wanted) ^{85 88 89 90 91 92}.

Règles du mainteneur : suivre les templates, joindre la sortie de `bash scripts/ctx-debug.sh` et le prompt exact, écrire des tests pour tout changement (TDD red-green-refactor), et ne PAS créer de nouveau fichier de test : compléter celui du domaine (`tests/adapters/<platform>.test.ts` , `tests/core/server.test.ts` , `tests/hooks/*.test.ts` , `tests/session/*.test.ts`) ¹⁷.

Bugs ouverts les plus discutés, bons candidats : #947 (le timeout de `ctx_batch_execute` ne borne pas l'indexation, 4 commentaires), #950 (`ctx_stats` incohérent, 5), #1024 (nettoyage de la DB qui supprime des sessions vivantes, 2), #1022 (snapshot de reprise qui ignore le budget d'octets, 2) ^{93 94 95 96 97 98 99 100}.

Si tu utilises Claude Code : #911 et #946 (6 et 3 commentaires) portent sur le classifieur auto-mode déclenché par le prompt injecté et demandent un opt-out, reproductibles sur ton poste ^{101 102 103 104}.

Avant d'ouvrir une PR, cherche un doublon parmi les 107 ouvertes (ex. #1138 tables HTML sans lignes, #1127 budgets echo par hôte, #1082 timeout en millisecondes) ^{60 75 105 106} ; la CI cible `main` et `next` ⁵⁴, mais le cache tronque CONTRIBUTING au début de « Submitting a Pull Request » : la branche de base des PR et la suite du processus sont à vérifier dans CONTRIBUTING.md ¹⁷.

Interdit par la politique « prose-style » : aucune directive de brièveté dans `src/server.ts` , `hooks/routing-block.mjs` ou `configs/*` , un test de régression dans `tests/core/server.test.ts` casse la CI sinon ; si ton changement touche la sortie d'un outil, joins un avant/après dans la PR ¹⁷.

\$ Vos 3 premières actions

1. Clone, bulde, teste : `git clone https://github.com/mksglu/context-mode.git && cd context-mode && npm install && npm run build && npm test` ; il faut node >= 22.5.0, `pretest` relance le build et vitest parcourt 255 fichiers de test ^{107 40 41 43 42 20 17}.
2. Branche ton clone sur Claude Code : `rm server.bundle.mjs` , symlink de `~/.claude/plugins/cache/context-mode/context-mode/<version>` vers le clone, override du seul hook `PreToolUse` dans `~/.claude/settings.json` , version `-dev` dans `package.json` , `src/server.ts` , `.claude-plugin/plugin.json` et `.claude-plugin/marketplace.json` , `npm run build` , `pkill -f "context-mode.*start.mjs"` , redémarre, puis `/context-mode:ctx-doctor` doit afficher ta version `-dev` ^{17 7}.
3. Prends #947 (le timeout de `ctx_batch_execute` ne borne pas l'indexation, 4 commentaires) ou #1024 (le nettoyage de la DB supprime des sessions vivantes, 2 commentaires) : test rouge d'abord dans le fichier existant du domaine (`tests/core/server.test.ts` pour les outils, `tests/store.test.ts` pour le store), jamais un nouveau fichier, puis `bash scripts/ctx-debug.sh` et un avant/après de la sortie dans la PR, après avoir vérifié qu'aucune des 107 PR ouvertes ne couvre déjà le sujet ^{93 94 97 98 60 17}.
4. Avant de pousser : `npm run typecheck` puis `npm test` , et aucune directive de brièveté dans `src/server.ts` , `hooks/routing-block.mjs` ou `configs/*` , le test de régression « prose-style policy (#482) » de `tests/core/server.test.ts` casse la CI sinon ^{20 17 54}.

NOTES

Sources

1. donnée collectée `repo.stars`
2. donnée collectée `repo.forks`
3. donnée collectée `repo.created_at`
4. donnée collectée `repo.license`
5. donnée collectée `risks.0.note`
6. donnée collectée `repo.description`
7. source du repo `readme.md`
8. donnée collectée `repo.topics`
9. donnée collectée `repo.open_issues`
10. donnée collectée `repo.pushed_at`
11. donnée collectée `repo.owner_type`
12. donnée collectée `repo.archived`
13. donnée collectée `repo.default_branch`
14. donnée collectée `tree.29.role`
15. donnée collectée `tree.23.role`
16. donnée collectée `tree.30.role`
17. source du repo `contributing.md`
18. donnée collectée `tree.27.role`
19. donnée collectée `tree.20.role`
20. source du repo `manifest.md`
21. source du repo `tree.md`
22. donnée collectée `tree.21.role`
23. donnée collectée `tree.32.role`
24. donnée collectée `tree.26.role`
25. source du repo `docs.md`
26. donnée collectée `tree.1.role`
27. donnée collectée `tree.3.role`
28. donnée collectée `tree.4.role`
29. donnée collectée `tree.11.role`
30. donnée collectée `tree.12.role`
31. donnée collectée `tree.6.role`
32. donnée collectée `entrypoints.0.path`
33. donnée collectée `entrypoints.2.path`
34. donnée collectée `entrypoints.0.why`
35. donnée collectée `entrypoints.1.path`
36. donnée collectée `entrypoints.1.why`
37. donnée collectée `entrypoints.2.why`
38. donnée collectée `entrypoints.4.why`
39. donnée collectée `entrypoints.3.why`
40. donnée collectée `build.install`
41. donnée collectée `build.test`
42. donnée collectée `tests.files`
43. donnée collectée `risks.4.note`
44. donnée collectée `risks.3.note`
45. donnée collectée `tree.19.role`
46. donnée collectée `tests.dir`
47. donnée collectée `tests.framework`
48. donnée collectée `build.run`
49. donnée collectée `build.ci.0.name`
50. donnée collectée `build.ci.0.path`
51. donnée collectée `build.ci.0.triggers.0`
52. donnée collectée `build.ci.0.triggers.1`
53. donnée collectée `build.ci.0.triggers.2`
54. source du repo `ci.md`
55. donnée collectée `activity.commits_per_week.1.count`
56. donnée collectée `activity.commits_per_week.10.count`
57. donnée collectée `activity.bus_factor`
58. donnée collectée `activity.contributors.0.commits`

59. donnée collectée `activity.contributors.1.commits`

60. donnée collectée `pulls.open`

61. donnée collectée `activity.commits_per_week.0.count`

62. donnée collectée `activity.commits_per_week.11.count`

63. donnée collectée `activity.last_commit`

64. donnée collectée `activity.contributors.2.commits`

65. donnée collectée `activity.contributors.3.commits`

66. donnée collectée `risks.2.note`

67. donnée collectée `releases.9.tag`

68. donnée collectée `releases.9.date`

69. donnée collectée `releases.0.tag`

70. donnée collectée `releases.0.date`

71. donnée collectée `issues.open`

72. donnée collectée `issues.closed_30d`

73. donnée collectée `pulls.merged_30d`

74. donnée collectée `roadmap.open_prs.0.updated_at`

75. donnée collectée `roadmap.open_prs.0.title`

76. donnée collectée `roadmap.open_prs.1.title`

77. donnée collectée `roadmap.open_prs.3.title`

78. donnée collectée `risks.3.level`

79. donnée collectée `risks.4.level`

80. donnée collectée `risks.0.level`

81. donnée collectée `risks.1.level`

82. donnée collectée `risks.2.level`

83. donnée collectée `business.codeowners`

84. donnée collectée `business.funding`

85. donnée collectée `issues.good_first`

86. donnée collectée `issues.hot.4.number`

87. donnée collectée `issues.hot.8.number`

88. donnée collectée `issues.by_label.enhancement`

89. donnée collectée `issues.by_label.bug`

90. donnée collectée `issues.hot.0.title`

91. donnée collectée `issues.hot.0.comments`

92. donnée collectée `roadmap.requests.0.labels.0`

93. donnée collectée `issues.hot.4.url`

94. donnée collectée `issues.hot.4.comments`

95. donnée collectée `issues.hot.3.url`

96. donnée collectée `issues.hot.3.comments`

97. donnée collectée `issues.hot.8.url`

98. donnée collectée `issues.hot.8.comments`

99. donnée collectée `issues.hot.9.url`

100. donnée collectée `issues.hot.9.comments`

101. donnée collectée `issues.hot.1.url`

102. donnée collectée `issues.hot.1.comments`

103. donnée collectée `issues.hot.5.url`

104. donnée collectée `issues.hot.5.comments`

105. donnée collectée `roadmap.open_prs.6.title`

106. donnée collectée `roadmap.open_prs.4.title`

107. donnée collectée `repo.url`